

Booking System with Dynamic Pricing and Coupon Validation

Scenario:

Build a booking system for events with dynamic pricing and coupon codes.

- Create a Booking model with fields: event_name, number_of_tickets, price_per_ticket, and total_price.
- Add a Coupon model with fields: code and discount_percentage.
- Create a form to allow users to:
 - o Select an event and enter the number of tickets.
 - o Enter a coupon code (if available).
- Calculate the total price dynamically based on the number of tickets and apply the coupon discount if valid.

Technologies to Use: Django forms, Django views, and Django ORM.

Criteria:

- Total price calculation should be dynamic and accurate.
- Coupons should be validated and applied correctly.
- Ensure input validations, such as non-negative ticket numbers and valid coupon codes.

Implementation:

Step 1: Set Up the Django Project and App

Create a Django project:

```
django-admin startproject bookingsystem
```

```
cd bookingsystem
```

Create the products app:

```
python manage.py startapp eventbooking
```

Add eventbooking to INSTALLED_APPS in settings.py.

Step 2: Define the Booking,Coupon,Event Models

In eventbooking/models.py:

```
from django.db import models
```

```
class Event(models.Model):
```

```
    name = models.CharField(max_length=255)
```

```
    price_per_ticket = models.DecimalField(max_digits=10, decimal_places=2)
```

```
    def __str__(self):
```

```
        return self.name
```

```
class Coupon(models.Model):
```

```
    code = models.CharField(max_length=50, unique=True)
```

```
    discount_percentage = models.DecimalField(max_digits=5, decimal_places=2) #e.g., 10.00  
    for 10%
```

```
    def __str__(self):
```

```
        return self.code
```

```
class Booking(models.Model):
```

```
    event_name = models.ForeignKey(Event, on_delete=models.CASCADE)
```

```
    number_of_tickets = models.PositiveIntegerField()
```

```
    price_per_ticket = models.DecimalField(max_digits=10, decimal_places=2)
```

```
    total_price = models.DecimalField(max_digits=10, decimal_places=2)
```

```
    def __str__(self):
```

```
        return f"Booking for {self.event_name.name} ({self.number_of_tickets} tickets) - Total:  
    ${self.total_price}"
```

Run migrations:

```
python manage.py makemigrations
```

```
python manage.py migrate
```

Step 3: Create forms

```

from django import forms

from .models import Event,Coupon,Booking

from django.core.exceptions import ValidationError


class BookingForm(forms.Form):

    event = forms.ModelChoiceField(queryset=Event.objects.all(), label="Select Event")

    number_of_tickets = forms.IntegerField(min_value=1, label="Number of Tickets")

    coupon_code = forms.CharField(max_length=50, required=False, label="Coupon Code")


    def clean_number_of_tickets(self):

        number_of_tickets = self.cleaned_data.get('number_of_tickets')

        if number_of_tickets <= 0:

            raise ValidationError("Number of tickets must be a positive integer.")

        return number_of_tickets


    def clean_coupon_code(self):

        code = self.cleaned_data.get('coupon_code')

        if code:

            try:

                Coupon.objects.get(code=code)

            except Coupon.DoesNotExist:

                raise ValidationError("Invalid coupon code.")

        return code

```

Step 4: Build API Views

```

from django.shortcuts import render

from django.db import models

from django import forms

from eventbooking.forms import BookingForm

```

```

from .models import Coupon,Event,Booking

def booking_view(request):

    total_price = None

    discount = 0

    form = BookingForm(request.POST or None)

    if request.method == "POST" and form.is_valid():

        event = form.cleaned_data['event']

        number_of_tickets = form.cleaned_data['number_of_tickets']

        coupon_code = form.cleaned_data.get('coupon_code')


        # Calculate the price per ticket

        price_per_ticket = event.price_per_ticket

        # Apply coupon discount if applicable

        if coupon_code:

            coupon = Coupon.objects.get(code=coupon_code)

            discount = coupon.discount_percentage / 100

        total_price = number_of_tickets * price_per_ticket * (1 - discount)

        # Save the booking

        Booking.objects.create(

            event_name=event,

            number_of_tickets=number_of_tickets,

            price_per_ticket=price_per_ticket,

            total_price=total_price

        )

        return render(request, "booking_success.html", {"total_price": total_price, "booking":
Booking.objects.last(), "applied_coupon": coupon if coupon_code else None})

coupons = Coupon.objects.all()

```

```
    return render(request, "booking_form.html", {"form": form, "total_price": total_price,
"coupons": coupons})
```

Step 6: Define URLs

In urls.py:

```
from django.contrib import admin

from django.urls import path,include

urlpatterns = [

    path('admin/', admin.site.urls),

    path('',include("eventbooking.urls")),

]
```

In eventbooking/urls.py:

```
from django.urls import path

from .views import booking_view

urlpatterns = [

    path('book/', booking_view, name='booking_view'),

]
```

Step 7: Register in admin.py

```
from django.contrib import admin

from .models import Event,Coupon,Booking

# Register your models here.

admin.site.register(Event)

admin.site.register(Coupon)

admin.site.register(Booking)
```

Step 8: Create a folder named templates under eventbooking

1. Add booking_form.html

```
<!DOCTYPE html>
```

```
<html lang="en">
```

```
<head>
```

```

<meta charset="UTF-8">

<meta name="viewport" content="width=device-width, initial-scale=1.0">

<title>Booking_Form</title>

</head>

<body>

    <h1>Book Event</h1>

    <form method="post">

        {% csrf_token %}

        {{ form.as_p }}

        <button type="submit">Book Now</button>

    </form>

    {% if total_price is not None %}

        <p>Total Price: {{ total_price }}</p>

    {% endif %}

    <h2>Available Events</h2>

    <ul>

        {% for event in form.fields.event.queryset %}

            <li>{{ event.name }} - ${{ event.price_per_ticket }} per ticket</li>

        {% endfor %}

    </ul>

</body>

</html>

```

2. Add booking_success.html

```

<!DOCTYPE html>

<html lang="en">

<head>

    <meta charset="UTF-8">

    <meta name="viewport" content="width=device-width, initial-scale=1.0">

```

```

<title>Booking Successful</title>
</head>
<body>
    <h1>Booking Successful</h1>
    <p>Your booking is confirmed!</p>
    <p><strong>Event:</strong> {{ booking.event_name.name }}</p>
    <p><strong>Number of Tickets:</strong> {{ booking.number_of_tickets }}</p>
    <p><strong>Price per Ticket:</strong> ${{ booking.price_per_ticket }}</p>
    <p><strong>Total Price:</strong> ${{ booking.total_price }}</p>
    {% if applied_coupon %}
        <p><strong>Coupon    Applied:</strong>    {{    applied_coupon.code    }}    ({{
applied_coupon.discount_percentage    }}% off)</p>
    {% endif %}
</body>
</html>

```

Step 8: To execute the project run the following commands

To create superuser use the the command: `python manage.py createsuperuser`

To run the project : `python manage.py runserver 8087` #any port number or can leave it blank

It will open the server at <http://127.0.0.1:8087/>

In <http://127.0.0.1:8087/admin/>

Data in Admin Panel:

Events:

Concert, Price Per Ticket: 20.00

Theater Play, Price Per Ticket: 25.00

Coupons:

DISCOUNT10, Discount Percentage: 10.00

SUMMER15, Discount Percentage: 15.00

Example of the Process: In <http://127.0.0.1:8087/book/>

Booking Form: The user selects Concert, enters 2 tickets, and enters DISCOUNT10.

Book Event

Select Event

Concert

Number of Tickets

2

Coupon Code (Optional)

DISCOUNT10

Book Now

Available Events

Concert - \$20.00 per ticket

Theater Play - \$25.00 per ticket

Activate Windows

Total Price Calculation:

Price per ticket: \$20

Number of tickets: 2

Coupon discount: 10% ($10/100 = 0.10$)

Total Price: $2 * 20 * (1 - 0.10) = 36.00$

Confirmation Page: Displays the booking confirmation with the applied coupon and the calculated total price.

Booking Successful

Your booking has been confirmed!

Event: Concert

Number of Tickets: 2

Price per Ticket: \$20.00

Total Price: \$36.00

Coupon Applied: DISCOUNT10 (10.00% off)